

Day 8 Assignment

Optimizing PostgreSQL Performance: Indexing, Stored Procedures and Replication

John Michael Bilbao
Techstart
August 18, 2026

Introduction

This assignment asks for five things done to a PostgreSQL database: a sample dataset restored, indexes added on the strength of EXPLAIN output, a stored procedure written, streaming replication configured with `pg_basebackup`, and the server's memory parameters tuned using the `pg_stat` views as evidence.

Everything below was run against PostgreSQL 16.14. Two clusters were built: a primary on port 5432 and a replica on 5433, streaming from it. The sample database is `dvdrental`, restored into a database named `exampledb`.

Every screenshot in this report is output captured from those two servers. A single script, `setup.sh`, performs all five steps in order and writes the output of each into `results/`, and the figures are rendered from those files rather than retyped, so no figure can claim something the database did not do. The script tears both clusters down and rebuilds them from scratch on each run, so all of it is repeatable.

Step 1: Setup and Preparation

The sample archive is the `dvdrental.zip` the assignment links to. `postgresqtutorial.com` serves it but refuses non-browser clients, so the script pulls the identical archive from a mirror. It is a `pg_restore` custom-format dump, not plain SQL, so it is loaded with `pg_restore` rather than `psql`.

```
>_ Windows PowerShell - restore the sample database

psql (PostgreSQL) 16.14

PS C:\pg\day8> pg_ctl -D primary start
PS C:\pg\day8> createdb exampledb
PS C:\pg\day8> pg_restore -d exampledb dvdrental.tar

 database | size
-----+-----
 exampledb | 16 MB
(1 row)
```

Figure 1: PostgreSQL 16.14, and exampledb after the restore.

The restore produces fifteen tables. The four this report touches are listed with their row counts, because the size of a table decides whether an index on it is worth having at all.

```

>_ psql - exampledb schema

=== tables ===
          List of relations
 Schema |      Name      | Type | Owner
-----+-----+-----+-----
 public | actor           | table | postgres
 public | address          | table | postgres
 public | category         | table | postgres
 public | city             | table | postgres
 public | country          | table | postgres
 public | customer         | table | postgres
 public | film            | table | postgres
 public | film_actor       | table | postgres
 public | film_category    | table | postgres
 public | inventory        | table | postgres
 public | language         | table | postgres
 public | payment          | table | postgres
 public | rental           | table | postgres
 public | staff            | table | postgres
 public | store            | table | postgres
(15 rows)

=== row counts of the tables this report touches ===
 table_name | count
-----+-----
 customer   |    599
 film       |   1000
 payment    |  14596
 rental     |  16044
(4 rows)

```

Figure 2: The fifteen tables of the dvdrental schema, and the row counts of the four used below. *payment*, at 14,596 rows, is the largest.

payment is the table both test queries read. It is worth looking at what it already has, because *dvdrental* ships with indexes on its foreign keys, and an index that already exists changes what a new one can be credited with.

```
>_ psql -ld payment
```

Table "public.payment"				
Column	Type	Collation	Nullable	Default
payment_id	integer		not null	nextval('payment_payment_id_seq'::regclass)
customer_id	smallint		not null	
staff_id	smallint		not null	
rental_id	integer		not null	
amount	numeric(5,2)		not null	
payment_date	timestamp without time zone		not null	

Indexes:

- "payment_pkey" PRIMARY KEY, btree (payment_id)
- "idx_fk_customer_id" btree (customer_id)
- "idx_fk_rental_id" btree (rental_id)
- "idx_fk_staff_id" btree (staff_id)

Foreign-key constraints:

- "payment_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES customer(customer_id) ON UPDATE CASCADE ON DELETE RESTRICT
- "payment_rental_id_fkey" FOREIGN KEY (rental_id) REFERENCES rental(rental_id) ON UPDATE CASCADE ON DELETE SET NULL
- "payment_staff_id_fkey" FOREIGN KEY (staff_id) REFERENCES staff(staff_id) ON UPDATE CASCADE ON DELETE RESTRICT

Figure 3: The payment table. Note `idx_fk_customer_id`, which already exists — this matters for Q2 below.

Step 2: Indexing Strategies

The assignment asks for the most frequently used queries to be identified. Rather than guess, the server was asked. `pg_stat_statements` was loaded at startup; it records every statement executed along with a call count and total time, which is what makes “frequently used” a measurable claim rather than an assumption. `pg_stat_user_tables` was used alongside it, because a large `seq_tup_read` relative to `seq_scan` is the symptom of a filter with no index behind it.

```
>_ psql - finding the queries worth indexing
```

=== busiest statements by total execution time ===

calls	total_ms	mean_ms	query
1	33.40	33.404	DO \$\$ BEGIN FOR i IN 1..40 LOOP PERFORM customer_id, amoun
40	31.50	0.788	SELECT customer_id, amount, payment_date FROM payment WHER
40	0.35	0.009	SELECT payment_id, amount, payment_date FROM payment WHERE

(3 rows)

=== tables being read sequentially (a missing-index symptom) ===

relname	seq_scan	seq_tup_read	idx_scan	avg_rows_per_seq_scan
payment	40	583840	40	14596

(1 row)

Figure 4: Where the time goes, and the sequential-scan evidence. `payment` is read end to end on every call, roughly 14,600 rows returned to find a few hundred.

Two queries came out of that, and they were chosen because they fail for different reasons:

	Query	Why it is slow
Q1	Payments taken on one particular day (a range over payment_date)	No index on payment_date at all, so the whole table is scanned
Q2	The ten most recent payments for one customer (filter, then sort)	An index on customer_id exists, but the sort by date is done afterwards in memory

The plans before indexing

Q1 is a plain sequential scan. The line worth reading is Rows Removed by Filter: 14,288 — PostgreSQL examined every row in the table and discarded all but 308 of them, touching 128 buffers to do it.

```
> _ psql - EXPLAIN Q1, no index

=== Q1: payments taken on one day ===

                                QUERY PLAN
-----
Seq Scan on payment (actual time=0.016..0.917 rows=308 loops=1)
  Filter: ((payment_date >= '2007-02-15 00:00:00'::timestamp without time zone) AND (payment_date < '2007-02-16 00:00:00'::timestamp without time zone))
  Rows Removed by Filter: 14288
  Buffers: shared hit=128
Planning:
  Buffers: shared hit=102
Planning Time: 0.318 ms
Execution Time: 0.959 ms
(8 rows)
```

Figure 5: Q1 with no index: Seq Scan, and 14,288 rows read only to be thrown away.

Q2 is more interesting, and it is the reason it was included. It is already using an index — idx_fk_customer_id, which shipped with the sample database — so the filter is not the problem. The problem is the two nodes above it: the rows come back in no particular order, so a Sort has to run before the Limit can take ten of them.

```
> psql - EXPLAIN Q2, no composite index

=== Q2: the ten most recent payments for one customer ===
                        QUERY PLAN
-----
Limit (actual time=0.056..0.057 rows=10 loops=1)
  Buffers: shared hit=11
  -> Sort (actual time=0.055..0.056 rows=10 loops=1)
        Sort Key: payment_date DESC
        Sort Method: top-N heapsort  Memory: 26kB
        Buffers: shared hit=11
    -> Bitmap Heap Scan on payment (actual time=0.024..0.029 rows=22 loops=1)
          Recheck Cond: (customer_id = 341)
          Heap Blocks: exact=3
          Buffers: shared hit=8
      -> Bitmap Index Scan on idx_fk_customer_id (actual time=0.020..0.020 rows=22 loops=1)
            Index Cond: (customer_id = 341)
            Buffers: shared hit=5

Planning:
  Buffers: shared hit=38
Planning Time: 0.104 ms
Execution Time: 0.077 ms
(17 rows)
```

Figure 6: Q2 with no composite index. The filter is already indexed; the top-N heapsort above it is the avoidable work.

The indexes

One index per query, each shaped to the query's problem. Q1 needs nothing more than a B-tree on the column it filters. Q2 needs a composite index whose leading column is the one being filtered and whose second column is stored in the direction the query asks to sort in — that way the index can satisfy the filter and the ordering at once, and the sort becomes unnecessary.

```
• 05_indexes.sql

3 -- Q1 filters on a range of payment_date, so a plain B-tree on that column is
4 -- enough.
5 CREATE INDEX IF NOT EXISTS idx_payment_payment_date
6   ON payment (payment_date);
7
8 -- Q2 filters on customer_id and then sorts by payment_date descending. A
9 -- composite index in that order lets the index satisfy both: the leading
10 -- column narrows to the customer, and because the second column is already
11 -- stored in the requested direction the sort disappears entirely.
12 CREATE INDEX IF NOT EXISTS idx_payment_customer_date
13   ON payment (customer_id, payment_date DESC);
14
15 ANALYZE payment;
```

sql05_indexes.sql PostgreSQL Spaces: 4 UTF-8

Figure 7: The two CREATE INDEX statements. Column order in a composite index is not arbitrary; it is what decides whether the sort can be skipped.

```
> _ psql - the two indexes, and what they cost

CREATE INDEX
CREATE INDEX
ANALYZE

=== indexes on payment ===
      indexname      |  size
-----+-----
idx_fk_customer_id   | 128 kB
idx_fk_rental_id     | 336 kB
idx_fk_staff_id       | 120 kB
idx_payment_customer_date | 464 kB
idx_payment_payment_date | 336 kB
payment_pkey         | 336 kB
(6 rows)
```

Figure 8: The indexes on payment afterwards. The two new ones cost 336 kB and 464 kB.

The plans after indexing

Q1 no longer reads the table. It finds the matching rows through `idx_payment_payment_date` and fetches only the pages that contain them, cutting buffers from 128 to 17.

```
> _ psql - EXPLAIN Q1, indexed

=== Q1: payments taken on one day ===
                                QUERY PLAN
-----
Bitmap Heap Scan on payment (actual time=0.025..0.079 rows=308 loops=1)
  Recheck Cond: ((payment_date >= '2007-02-15 00:00:00'::timestamp without time zone) AND (payment_date < '2007-02-16 00:00:00'::timestamp without time zone))
  Heap Blocks: exact=15
  Buffers: shared hit=17
-> Bitmap Index Scan on idx_payment_payment_date (actual time=0.013..0.013 rows=308 loops=1)
    Index Cond: ((payment_date >= '2007-02-15 00:00:00'::timestamp without time zone) AND (payment_date < '2007-02-16 00:00:00'::timestamp without time zone))
    Buffers: shared hit=2
Planning:
  Buffers: shared hit=146 read=4
Planning Time: 0.455 ms
Execution Time: 0.125 ms
(11 rows)
```

Figure 9: Q1 indexed: Bitmap Index Scan, 15 heap blocks instead of the whole table.

Q2's plan is now four lines instead of nine. The Sort and the Bitmap Heap Scan are both gone, replaced by a single Index Scan: because the index already holds the rows for that customer in descending date order, the Limit takes the first ten and stops.

```

> _ psql - EXPLAIN Q2, indexed

=== Q2: the ten most recent payments for one customer ===
                        QUERY PLAN
-----
Limit (actual time=0.031..0.035 rows=10 loops=1)
  Buffers: shared hit=5 read=2
  -> Index Scan using idx_payment_customer_date on payment (actual time=0.031..0.033 rows=10 loops=1)
       Index Cond: (customer_id = 341)
       Buffers: shared hit=5 read=2
Planning:
  Buffers: shared hit=37
Planning Time: 0.088 ms
Execution Time: 0.043 ms
(9 rows)

```

Figure 10: Q2 indexed. The Sort node has disappeared entirely — the index supplies the order.

Measuring it properly

A single EXPLAIN ANALYZE of a query that runs in well under a millisecond is mostly noise; running the same statements repeatedly during this work produced execution times that varied by a factor of three in both directions. So each query was run 200 times and the mean that pg_stat_statements recorded was compared instead.

```

> _ psql - 200 runs of each query, before and after

--- BEFORE the indexes ---
=== mean execution time over 200 runs each ===
      test_query      | calls | mean_ms | total_ms | blocks_per_call
-----+-----+-----+-----+-----
Q1 (date range)      |    200 |  0.7649 |   152.98 |          128.0
Q2 (customer, recent first) |    200 |  0.0068 |     1.36 |             5.0
(2 rows)

--- AFTER the indexes ---
=== mean execution time over 200 runs each ===
      test_query      | calls | mean_ms | total_ms | blocks_per_call
-----+-----+-----+-----+-----
Q1 (date range)      |    200 |  0.0438 |     8.76 |          17.0
Q2 (customer, recent first) |    200 |  0.0029 |     0.57 |             4.0
(2 rows)

```

Figure 11: 200 runs of each query, before and after. The block counts are deterministic; the means are stable enough to compare.

	Mean before	Mean after	Improvement	Blocks per call
Q1	0.7649 ms	0.0438 ms	17.5x faster	128 → 17
Q2	0.0068 ms	0.0029 ms	2.3x faster	5 → 4

Q1 is the large win, at 17.5x, because it went from having no usable index to having one. Q2's 2.3x is the smaller number and the more honest one: an index was already helping it, so the composite index only removed the sort. On 14,596 rows that saves very little wall-clock time. The reason it is still worth doing is that the sort's cost grows with the number of rows matching the customer while the index scan's does not, so the gap widens as the table grows — and this table is the one that grows fastest in a rental business.

Step 3: Stored Procedures

Two objects were written, deliberately one of each kind, because the distinction is the interesting part of this step. PostgreSQL has had both CREATE FUNCTION and CREATE PROCEDURE since version 11, and they are not interchangeable.

add_customer is a FUNCTION. It inserts a customer and returns the generated customer_id, which is what a caller needs. Before inserting it checks two things: that the address looks like an email at all, and that it is not already registered. Both checks raise with a specific SQLSTATE rather than a bare error, so a caller can tell a duplicate from a malformed input without parsing the message text.

```
• 07_functions.sql

7 CREATE OR REPLACE FUNCTION add_customer(
8     p_store_id smallint,
9     p_first_name text,
10    p_last_name text,
11    p_email text,
12    p_address_id smallint)
13 RETURNS integer
14 LANGUAGE plpgsql
15 AS $$
16 DECLARE
17     v_customer_id integer;
18 BEGIN
19     IF p_email IS NULL OR position('@' IN p_email) = 0 THEN
20         RAISE EXCEPTION 'not an email address: %', p_email
21         USING ERRCODE = 'check_violation';
22     END IF;
23
24     IF EXISTS (SELECT 1 FROM customer WHERE lower(email) = lower(p_email)) THEN
25         RAISE EXCEPTION 'email already registered: %', p_email
26         USING ERRCODE = 'unique_violation';
27     END IF;
28
29     INSERT INTO customer (store_id, first_name, last_name, email,
30                          address_id, activebool, create_date, active)
31     VALUES (p_store_id, p_first_name, p_last_name, lower(p_email),
32             p_address_id, true, current_date, 1)
33     RETURNING customer_id INTO v_customer_id;
34
35     RAISE NOTICE 'created customer % (%)', v_customer_id, lower(p_email);
36     RETURN v_customer_id;
37 END;
38 $$;
```

sql07_functions.sql PostgreSQL Spaces: 4 UTF-8

Figure 12: `add_customer`. The validation is in the database, so it holds no matter which application does the inserting.

`deactivate_customer` is a PROCEDURE. It returns nothing, and it commits its own transaction. That is the capability a procedure has and a function does not: a function runs inside the caller's transaction and cannot commit. It also checks `ROW_COUNT` after the `UPDATE` and raises if nothing matched, so asking to deactivate a customer who is already inactive is an error rather than a silent no-op.

```
• 07_functions.sql

40 -- deactivate_customer is a PROCEDURE. It returns nothing, and it commits its
41 -- own work: that transaction control is the thing a procedure can do and a
42 -- function cannot.
43 CREATE OR REPLACE PROCEDURE deactivate_customer(p_customer_id integer)
44 LANGUAGE plpgsql
45 AS $$
46 DECLARE
47     v_rows integer;
48 BEGIN
49     UPDATE customer
50     SET activebool = false, active = 0, last_update = now()
51     WHERE customer_id = p_customer_id AND activebool;
52
53     GET DIAGNOSTICS v_rows = ROW_COUNT;
54
55     IF v_rows = 0 THEN
56         RAISE EXCEPTION 'no active customer with id %', p_customer_id
57         USING ERRCODE = 'no_data_found';
58     END IF;
59
60     COMMIT;
61     RAISE NOTICE 'customer % deactivated and committed', p_customer_id;
62 END;
63 $$;
```

sql07_functions.sql PostgreSQL Spaces: 4 UTF-8

Figure 13: deactivate_customer. The COMMIT on line 60 is the reason this is a procedure.

```
> _ psql - the function and the procedure exist

CREATE FUNCTION
CREATE PROCEDURE
=== what was created ===
      name          | kind      | returns
-----+-----+-----
add_customer        | function  | integer
deactivate_customer | procedure |
(2 rows)
```

Figure 14: Both objects as PostgreSQL sees them. Note the empty returns column for the procedure.

Both were then tested with five calls, three of which are supposed to fail. Testing only the path that works would not show that the validation does anything.

```
> _ psql - exercising both, failures included

=== 1. add a customer (expect: a new id) ===
psql:08_functions_test.sql:6: NOTICE:  created customer 600 (john.bilbao@example.com)
new_customer_id
-----
        600
(1 row)

=== 2. the same email again (expect: rejected, unique_violation) ===
psql:08_functions_test.sql:11: ERROR:  email already registered: JOHN.BILBAO@EXAMPLE.COM
CONTEXT:  PL/pgSQL function add_customer(smallint,text,text,text,smallint) line 11 at RAISE

=== 3. a malformed email (expect: rejected, check_violation) ===
psql:08_functions_test.sql:16: ERROR:  not an email address: not-an-email
CONTEXT:  PL/pgSQL function add_customer(smallint,text,text,text,smallint) line 6 at RAISE

=== 4. deactivate that customer (expect: committed) ===
psql:08_functions_test.sql:25: NOTICE:  customer 600 deactivated and committed
CALL

=== 5. deactivate the same customer again (expect: rejected) ===
psql:08_functions_test.sql:29: ERROR:  no active customer with id 600
CONTEXT:  PL/pgSQL function deactivate_customer(integer) line 12 at RAISE

=== the resulting row ===
 customer_id | first_name | last_name | email | activebool | create_date
-----+-----+-----+-----+-----+-----
        600 | John      | Bilbao   | john.bilbao@example.com | f          | 2026-08-18
(1 row)
```

Figure 15: All five calls. The successes report through RAISE NOTICE; the three rejections each carry the SQLSTATE they were raised with.

The results are as designed: the first call returns customer_id 600; the same address in different case is rejected as a duplicate, which confirms the comparison is case-insensitive; a string with no @ is rejected; the deactivation commits; and repeating it is refused because no active customer with that id remains. The final row shows activebool has flipped to f, so the procedure's work was committed and not rolled back.

One thing this step ran into: CALL will not accept a subquery as an argument, so fetching the id with CALL deactivate_customer((SELECT ...)) fails to parse. The id is fetched into a psql variable with \gset first and the variable passed in.

Step 4: Replication Setup

The replica is a second cluster on the same host, listening on port 5433 and streaming from the primary on 5432. pg_basebackup was run with -R, which writes standby.signal and the primary_conninfo into the new data directory so the copy comes up as a standby without any further configuration, and with -C -S replica1 to create a replication slot. The slot is the part worth having: it makes the primary retain WAL the replica has not consumed yet, so a replica that falls behind or disconnects can catch up instead of being unrecoverable.

>_ Windows PowerShell - pg_basebackup

```
pg_basebackup: initiating base backup, waiting for checkpoint to complete
pg_basebackup: checkpoint completed
pg_basebackup: write-ahead log start point: 0/2000028 on timeline 1
pg_basebackup: starting background WAL receiver
pg_basebackup: created replication slot "replica1"
    64/39970 kB (0%), 0/1 tablespace (...b/pgsql/day8/replica/global/2694)
  39980/39980 kB (100%), 0/1 tablespace (...l/day8/replica/global/pg_control)
  39980/39980 kB (100%), 1/1 tablespace
pg_basebackup: write-ahead log end point: 0/203C6F8
pg_basebackup: waiting for background process to finish streaming ...
pg_basebackup: syncing data to disk ...
pg_basebackup: renaming backup_manifest.tmp to backup_manifest
pg_basebackup: base backup completed
```

Figure 16: `pg_basebackup` copying the cluster and creating the slot. 39,980 kB, one tablespace.

Because the copy is byte-for-byte, it includes the primary's `postgresql.conf` and therefore its port. Starting it unchanged would collide with the primary, so the replica's port is overridden to 5433 afterwards.

>_ psql - the replica identifies as a replica

```
PS C:\pg\day8> pg_ctl -D replica start          # replica listening on 5433

--- is this server a replica? ---
  in_recovery | replayed
-----+-----
t             | 0/3000060
(1 row)
```

Figure 17: The replica confirms it is in recovery — which is what a streaming standby is permanently in.

```

> _ psql - pg_stat_replication on the primary

=== pg_stat_replication (run on the primary) ===
 application_name | state | sync_state | sent_lsn | replay_lsn | replay_lag_bytes
-----+-----+-----+-----+-----+-----
 replical         | streaming | async      | 0/3000060 | 0/3000060 | 0
(1 row)

=== replication slot ===
 slot_name | slot_type | active
-----+-----+-----
 replical  | physical  | t
(1 row)

```

Figure 18: `pg_stat_replication` on the primary: streaming, asynchronous, and caught up to the byte.

`sent_lsn` and `replay_lsn` are equal, so the replica has replayed everything sent to it. `sync_state` is `async`, which is the default: the primary commits without waiting for the replica to confirm. That is the right trade for a read replica and the wrong one for zero-loss failover, which would need `synchronous_commit` and a synchronous standby, at the cost of every commit on the primary waiting for a network round trip.

A status view showing streaming is not by itself proof that data arrives, so it was tested directly: insert on the primary, read on the replica, then attempt a write on the replica.

```
> _ psql - the replica streams, and refuses writes

--- on the PRIMARY (5432): insert a row ---
category_id |      name
-----+-----
          17 | Replication Test
(1 row)

INSERT 0 1

--- on the REPLICA (5433): the row arrived ---
category_id |      name
-----+-----
          17 | Replication Test
(1 row)

--- on the REPLICA (5433): try to write to it ---
ERROR:  cannot execute INSERT in a read-only transaction
```

Figure 19: The row inserted on 5432 read back from 5433, and the replica refusing to be written to.

The row appears on the replica, and the attempted insert there is refused with “cannot execute INSERT in a read-only transaction”. Both halves matter: the first shows replication works, the second shows the replica is genuinely read-only and cannot silently diverge from the primary.

Step 5: Performance Tuning

The two views the assignment names answer different questions. `pg_stat_user_tables` says whether a table is being read sequentially or through an index; `pg_stat_user_indexes` says whether each index is actually earning its place.

```
> _ psql - pg_stat_user_tables

=== pg_stat_user_tables: sequential vs index reads ===
 relname  | seq_scan | seq_tup_read | idx_scan | n_live_tup
-----+-----+-----+-----+-----
 customer |         4 |         2399 |         2 |          1
 film     |         0 |          0   |         0 |          0
 payment  |        243 |       3546828 |        645 |       14596
 rental   |         0 |          0   |         0 |          0
(4 rows)
```

Figure 20: `pg_stat_user_tables`. *payment* now has more index scans than sequential ones.

```
> _ psql - pg_stat_user_indexes

=== pg_stat_user_indexes: is each index actually being used? ===
 index_name | scans | tuples_read | size
-----+-----+-----+-----
 idx_fk_customer_id |    241 |         5302 | 128 kB
 idx_payment_payment_date |    203 |        61910 | 336 kB
 idx_payment_customer_date |    201 |         2010 | 464 kB
 idx_fk_rental_id |         0 |          0 | 336 kB
 idx_fk_staff_id |         0 |          0 | 120 kB
 payment_pkey |         0 |          0 | 336 kB
(6 rows)
```

Figure 21: `pg_stat_user_indexes`. *Both new indexes are being used — and three others are not.*

The second view produced the most useful finding in this report, and it is not about the queries that were optimised. Both new indexes are being used, roughly 200 scans each, which confirms the planner chose them rather than merely being able to. But three indexes on `payment` have zero scans: `idx_fk_rental_id`, `idx_fk_staff_id`, and `payment_pkey`. An unused index is not free — every `INSERT`, `UPDATE` and `DELETE` on the table has to maintain it, and it occupies cache that could hold table data. Together those three hold 792 kB doing nothing for this workload.

They should not be dropped on this evidence, though. The counters were reset at the start of this exercise, so “zero scans” means zero during a short synthetic workload of two queries, not zero in production; and `payment_pkey` enforces the primary key, so it earns its place regardless of whether anything reads it. The correct conclusion is that these are the candidates to watch over a representative period, not that they are dead weight.


```
> _ psql - cache hit ratio

=== cache hit ratio (how often a read was served from shared_buffers) ===
 relname  | disk_blocks | cache_blocks | cache_hit_pct
-----+-----+-----+-----
 customer |           0 |           45 |          100.00
    film  |           0 |            0 |
 payment  |           0 |          35374 |          100.00
    rental |           0 |            0 |
(4 rows)
```

Figure 22: `pg_statio_user_tables`: every read was served from `shared_buffers`.

The cache hit ratio is 100%, which is worth interpreting rather than celebrating. `exampledb` is 16 MB and the default `shared_buffers` is 128 MB, so the entire database fits in memory several times over. It also explains why the improvements in Step 2 are measured in microseconds: no query in this exercise ever waited on a disk. On a database larger than RAM the same index changes would show much larger gains, because the buffer reduction of 128 blocks to 17 would be 111 fewer trips to storage rather than 111 fewer memory lookups.

Configuration

```
> _ psql - configuration as installed

detected RAM: 31560 MB across 8 cores
target: shared_buffers 7890MB (25%), effective_cache_size 23670MB (75%)

--- defaults ---
      name      | value | source
-----+-----+-----
effective_cache_size | 4096 MB | default
maintenance_work_mem | 64 MB | default
random_page_cost | 4 | default
shared_buffers | 128 MB | configuration file
work_mem | 4096 kB | default
(5 rows)
```

Figure 23: The defaults. `shared_buffers` of 128 MB on a machine with 31 GB.

The defaults are deliberately conservative so that PostgreSQL starts on almost anything, which means they are wrong for any real server. The values below were sized from the machine actually being used — 31,560 MB of RAM across 8 cores — rather than copied from a guide.

Parameter	Default	Set to	Reasoning
shared_buffers	128 MB	7890 MB	25% of RAM, the conventional starting point; PostgreSQL relies on the OS page cache for the rest
effective_cache_size	4 GB	23 GB	75% of RAM. Allocates nothing — it tells the planner how much caching to assume, which makes it favour index scans
work_mem	4 MB	32 MB	Per sort or hash, not per connection, so it multiplies; raised modestly to keep sorts out of temporary files
maintenance_work_mem	512 MB	512 MB	Used by CREATE INDEX and VACUUM, which run few at a time, so it can be far larger than work_mem
random_page_cost	4	1.1	The default assumes spinning disks. On SSD a random read costs nearly what a sequential one does, and leaving it at 4 makes the planner avoid indexes it should use

shared_buffers is the one that needs a restart rather than a reload, because it allocates a shared memory segment at startup. The other four would have taken a reload.

```
>_ Windows PowerShell - after tuning and restart

PS C:\pg\day8> psql -c "ALTER SYSTEM SET shared_buffers = '7890MB'" # +4 more
PS C:\pg\day8> pg_ctl -D primary -m fast restart # shared_buffers needs a restart

--- after ---
      name      | value  | source
-----+-----+-----
effective_cache_size | 23 GB  | configuration file
maintenance_work_mem | 512 MB | configuration file
random_page_cost    | 1.1    | configuration file
shared_buffers      | 7890 MB | configuration file
work_mem            | 32 MB  | configuration file
(5 rows)

--- the replica reconnected on its own after the restart ---
 application_name | state  | sync_state
-----+-----+-----
replica1         | streaming | async
(1 row)
```

Figure 24: The five parameters in effect after the restart, and the replica back in streaming state.

The second half of that screenshot is there on purpose. Restarting the primary drops the replica's connection, and it reconnects on its own when it next retries — the interval is wal_retrieve_retry_interval, 5 seconds by default. Querying pg_stat_replication sooner than that

returns no rows and looks exactly like replication having broken, which is what happened on the first attempt at this capture.

Findings and Recommendations

The indexes did what they were meant to. Q1 went from reading all 14,596 rows to reading 308, 17.5x faster over 200 runs with buffer reads down from 128 to 17. Q2 kept the same speed to within a microsecond or two but lost a Sort node, which is the change that will matter as the table grows.

The clearer lesson is about what the numbers do and do not show. This database is 16 MB and fits entirely in cache, so every measurement here is of CPU and memory work, never of disk. That makes the millisecond figures the least transferable thing in this report and the buffer counts and plan shapes the most: those are deterministic, and on a database that does not fit in RAM they are what turns into real time.

Recommendations, in the order worth acting on:

- Leave `pg_stat_statements` enabled permanently. Every indexing decision in this report came out of it, and it costs very little.
- Re-check `pg_stat_user_indexes` after a representative period, not after a synthetic workload. Three indexes on payment show zero scans here, which is a question to investigate rather than an answer.
- Apply the same configuration to the replica. It was cloned before the parameters were changed, so it is still running the defaults — which is a problem the moment it is promoted or used to serve reads.
- Consider making the replica synchronous only if losing recently committed transactions on failover is unacceptable, and accept that every commit on the primary then waits for it.
- Do the indexing work again against production-sized data. The plans chosen here are correct, but the planner's choices depend on table statistics, and 14,596 rows is small enough that it will sometimes prefer a sequential scan on principle.

A note on presentation: the shell prompts in the figures are shown as PowerShell to match the earlier assignments in this series. The `psql` output is unedited apart from shortening the absolute cluster paths to the short names used in this report, which `setup.sh` does to its own captures so that the shortening is applied consistently rather than by hand.